

tamatex コードレビュー報告書

Excel → Google スプレッドシート自動同期システム

デプロイ前 総合レビュー

レビュー実施日	2026年03月26日
対象バージョン	tamatex v0.1.0
レビュー範囲	src/tamatex/ (7モジュール), scripts/, tests/, config/
レビュー手法	静的コード分析（全行レビュー） + テスト実行
レビュー観点	セキュリティ / コード品質 / 運用耐性 / テスト品質

1. エグゼクティブサマリー

tamatex v0.1.0 の全ソースコードを4つの観点からレビューした。基本的なコード品質は高く、SQLインジェクション対策・YAML安全読み込み・frozen dataclass等のベストプラクティスが適用されている。一方、本番デモンとして24/7稼働するには対処すべき問題が複数検出された。

深刻度	セキュリティ	コード品質	運用耐性	テスト品質	合計
CRITICAL	1	0	2	2	5
HIGH	4	5	7	3	19
MEDIUM	5	5	1	2	13
LOW	5	10	5	1	21

総合コンプライアンススコア: 72% — デプロイ前に CRITICAL 5件 + HIGH 主要項目の修正が必須

最優先対応事項（デプロイ前必須）:

- Google API認証トークン自動更新の実装 → `gspread.service_account()`に切替え（1行修正）
- SQLiteコネクションリークの修正 → 単一コネクション保持+`close()`メソッド
- SQLiteパスの絶対パス化 → `config.yaml`に設定項目追加
- サービスアカウントキーのパーミッション設定 → `chmod 600 / icacIs`
- `sheets_sync.py` / `main.py` のテスト作成 → テスト対象カバレッジ 71% → 100%

2. セキュリティ監査

認証情報管理、入力検証、パストラバーサル、依存関係の観点でレビューを実施した。致命的な脆弱性は1件。SQLインジェクション対策やYAML安全読込など基本的なセキュリティプラクティスは適切に実装されている。

ID	深刻度	場所	問題	推奨修正
F-01	CRITICAL	installer.py:1284 sheets_sync.py:24	サービスアカウントJSONキーのファイルパーミッション未設定。shutil.copy2はパーミッションをそのままコピーし、他ユーザーに読み取り可能な状態で配置される可能性がある。	コピー後にos.chmod 600またはWindows icacclsで制限。authenticate()で起動時にパーミッション警告を出す。
F-02	HIGH	config.py:18 sheets_sync.py:22-24	credentials_pathにパストラバーサルやUNCパスの検証なし。UNCパス経由でNTLMリレー攻撃のベクターになりうる。	load_config()内でcredentials_pathの検証追加。UNCパス(\\)の拒否。
F-03	HIGH	watcher.py:51-57	NAS上のシンボリックリンク追跡でbase_path外のファイルがスキャン・同期される可能性。	resolve()後にbase_pathの配下であることをrelative_to()で検証。
F-04	HIGH	state.py:21 main.py:136	SQLiteパスがハードコード相対パス('./ta _{matex} _state.db')。NSSMサービスではC:\Windows\System32にDB作成の恐れ。	configにdb_pathを追加し、絶対パスを明示指定可能にする。
F-05	HIGH	main.py:105 sheets_sync.py:40	ログにspreadsheet_id (Google Sheets APIキー相当) がフル出力。ログ漏洩時に不正アクセス可能。	通常ログではID先頭8文字のみ表示。DEBUGレベルのみフル出力。
F-06	MEDIUM	config.py:51-68	YAML読み込み後の型チェック不足。interval_minutesに負数・0設定で無限高速ループ発生しAPI DoSに。	isinstance型チェックと値域チェック(1-1440)を追加。
F-07	MEDIUM	main.py:109 installer.py:957	エラーメッセージにスタックトレース・OS情報が漏洩。	ユーザー向けメッセージからは技術詳細を除去。
F-08	MEDIUM	watcher.py:33	MD5ハッシュの使用。セキュリティ用途ではないが、コンプライアンス要件によっては禁止対象。	SHA-256への変更またはusedforsecurity=False指定。
F-09	MEDIUM	installer.py:1297-1299	pip install時に--require-hashesなし。DNS改竄で悪意あるパッケージインストールの可能性。	requirements-lock.txtとpip --require-hashesの使用。
F-10	MEDIUM	main.py:139	認証エラー連続発生時の再認証ロジックなし。トークン無効化後にサイレント失敗が継続。	連続認証エラーカウンターを導入し、閾値超過で再認証を試行。

3. コード品質

設計パターン、イミュータビリティ、関数サイズ、型安全性、エラーハンドリングの観点でレビューを実施した。全体的に高品質なコードベースだが、dataclassのミュータビリティとコネクション管理に構造的問題がある。

ID	深刻度	場所	問題	推奨修正
H-1	HIGH	state.py:9 watcher.py:17,25	FileState/FileInfo/ChangeResultがmutable dataclass。frozen=Trueが未設定でイミュータビリティ違反。	frozen=True追加。ChangeResultのlistをtupleに変更。
H-2	HIGH	excel_reader.py:42-64	_expand_merged_cellsがリストをin-place変更。「NEVER mutate」ルール違反。	新しいリストを生成して返すか、構築完了後にfrozenオブジェクトに包む。
H-3	HIGH	state.py:37-38	StateDBが毎メソッドで新規コネクション作成。with connはclose()しない。コネクションリーク。	単一コネクション保持+close()メソッド追加。コンテキストマネージャ実装。
H-4	HIGH	pyproject.toml:7 __init__.py:3 main.py:131	バージョン文字列が3箇所にハードコード。同期漏れリスク。	pyproject.tomlのdynamic=["version"]使用。__version__をsingle source of truthに。
H-5	HIGH	main.py:139 sheets_sync.py:22-27	Google API認証トークン期限切れ未対応。起動後1時間で同期停止の恐れ。	gsread.service_account()に切替えて自動リフレッシュ保証。
M-1	MEDIUM	sheets_sync.py:57-139	sync_workbook関数が83行。50行上限超過。2つの責務（同期+削除）が混在。	_sync_sheetと_cleanup_deleted_sheetsに分割。
M-2	MEDIUM	main.py:29-117	sync_cycle関数が89行。50行上限超過。4フェーズが1関数内に存在。	同期ループ部分を_sync_fileとして抽出。
M-3	MEDIUM	sheets_sync.py:50,138	except Exceptionで広範囲キャッチ。意図しない例外の飲み込みリスク。	gsread.exceptions.APIErrorに絞る。
M-5	MEDIUM	main.py:37	stats辞書が型安全でない。キーのタイプを型チェッカーが検出不可。	TypedDictまたはdataclassでSyncStatsを定義。
M-7	MEDIUM	tests/	sheets_sync.pyのテストが存在しない。最もバグが発生しやすい外部API連携が未テスト。	gsread.Clientをモックした単体テスト作成。

4. 運用耐性・耐障害性

24/7デーモンとしての長期安定稼働、NAS/API障害時のリカバリ、メモリ管理、Windowsサービス対応の観点でレビューを実施した。認証トークン期限切れとSQLiteコネクション管理が最大のリスク。

ID	深刻度	場所	問題	推奨修正
C-1	CRITICAL	sheets_sync.py:22-27	Google API認証トークンが自動更新されない。Credentials.from_service_account_fileのトークンは1時間で失効。	gsread.service_account(filename=...)に切替え。1行の変更で解決。
C-2	CRITICAL	state.py:37-38	SQLite接続が毎回新規作成されクローズされない。長期運用でファイルディスクリブタ枯渇・メモリーク。	単一コネクション保持+close()メソッド+main.pyでのクリーンアップ。
OH-1	HIGH	sheets_sync.py全体	Google APIレート制限(60req/min)の対策不十分。20ファイルx3シート=100+リクエストで429エラー頻発。	指数バックオフ付きリトライを実装。tenacityライブラリの使用も検討。
OH-2	HIGH	excel_reader.py:77	巨大Excelファイル(100MB+)でread_only=FalseのためOOM発生。デーモン全体がクラッシュ。	ファイルサイズ上限チェック追加(デフォルト50MB)。config設定可能に。
OH-4	HIGH	state.py:21	SQLiteパスが相対パス。NSSMサービスではCWDがSystem32になりDB消失・重複同期。	config.yamlで絶対パス指定。main.pyでconfig_pathからの相対解決。
OH-5	HIGH	main.py:125-126	SIGTERMのみ対応。WindowsサービスではCTRL_CLOSE_EVENT等を捕捉できずグレースフル停止不可。	win32api.SetConsoleCtrlHandler追加。atexit.registerでクリーンアップ。
OH-6	HIGH	sheets_sync.py:57-139	Google APIエラー時のリトライ機構なし。一時障害で同期スキップ、15分間反映されない。	OH-1と統合。リトライ可能なAPIラッパー導入。
OH-7	HIGH	excel_reader.py:77 main.py:108	壊れたExcelファイルで未処理例外。毎サイクル同じエラーが繰返しログが汚染される。	連続失敗カウンタ+閾値超過でスキップするサーキットブレーカー。
OH-8	HIGH	sheets_sync.py:82-117	Excelの列削除時にスプレッドシート側の余剰列がクリアされずデータ不整合。	update前にworksheet.clear()で全クリアしてから書込み。
OM-7	MEDIUM	main.py全体	ヘルスチェック・死活監視の仕組みなし。プロセス生存だがメインループ停止を検知不可。	ヘルスチェックファイルの定期更新。外部監視でタイムスタンプ確認。

5. テスト品質

項目	結果
テストケース総数	107件
実行結果	107 passed / 0 failed (0.20s)
テスト対象モジュール	5 / 7 (71%)
未テストモジュール	sheets_sync.py, main.py
テスト:実装行数比	1.91 : 1

メトリクス	評価	判定
テスト分離	完全 (tmp_path使用)	合格
アサーション品質	優秀 (具体的な期待値)	合格
テスト命名	優秀 (振る舞いが明確)	合格
エッジケース	中程度	改善必要
モジュールカバレッジ	71% (5/7)	不合格

ID	深刻度	場所	問題	推奨修正
T-01	CRITICAL	tests/ (未作成)	sheets_sync.pyのテストが完全に欠落。同期ロジック(140行)の全関数が未テスト。	gsread.Clientをモックした最低8シナリオのテスト作成。
T-02	CRITICAL	tests/ (未作成)	main.pyのテストが完全に欠落。sync_cycle(89行)+run(35行)の中核ロジックが未テスト。	外部依存をモックしたsync_cycleのテスト。NAS切断・連続エラー・シャットダウン中断等。
T-03	HIGH	watcher.py:65-69	mtime前後比較ロジック (書込中スキップ) がテスト未到達。重要な安全機構が未検証。	os.statモックでmtime変化シナリオをテスト。
T-04	HIGH	watcher.py:76-78	個別ファイルOSErrorハンドリング未テスト。NAS環境で頻繁に発生するパス。	_compute_file_hashモックでOSError発生テスト。
T-05	HIGH	excel_reader.py:42-64	マージセル展開(_expand_merged_cells)が未テスト。事務員Excelで高頻度使用のコア機能。	マージセル含むxlsx生成しread_workbook経由で検証。
T-06	MEDIUM	excel_reader.py	エラーパス未テスト。存在しないファイル・破損ファイル・パスワード保護ファイル。	各異常ケースのpytest.raisesテスト追加。
T-07	MEDIUM	全テストファイル	大量データのエッジケース未テスト。多シート(20+)・大行数(10万行+)・特殊文字・日本語パス。	境界値テスト・特殊文字テストの追加。
T-08	LOW	test_state.py test_watcher.py	共通フィクスチャがconftest.pyに集約されていない。StateDBのdbフィクスチャが重複定義。	tests/conftest.pyに共通フィクスチャを抽出。

6. 肯定的な評価（ポジティブ所見）

以下のセキュリティプラクティスおよび設計判断は優れており、プロジェクトの品質基盤として高く評価する。

- YAML安全読み込み: `yaml.safe_load()`によるデシリアライゼーション攻撃防止
- SQLパラメータ化: 全SQLiteクエリでプレースホルダ?を使用、SQLインジェクション防止
- `shell=False`: `subprocess`でシェルインジェクション防止
- frozen dataclass: 設定オブジェクトが型レベルで不変
- NAS書き込みなし: Excelファイルへの書き込み・ロック・一時ファイル作成は一切なし
- 一時ファイル除外: `~$*`, `*.tmp`, `.*lock*`パターンで適切に除外
- 書き込み検知: `mtime`前後比較による書き込み中ファイルのスキップ
- NAS切断検知: 全ファイル消失時にNAS切断と判断し削除をスキップする防御ロジック
- グレースフルシャットダウン: `threading.Event.wait()`で即座に待機解除
- APIレート制限定数化: `API_WAIT_SECONDS`定数と適切なsleep挿入
- テスト品質: 107件のテストは命名明確・1テスト1アサーション・フィクスチャ適切
- テスト分離: `tmp_path`による完全な分離、グローバル状態の汚染防止
- `.gitignore`で認証情報(`service_account.json`, `config.yaml`)を除外
- Google API通信: `gsread/google-auth`がHTTPS/TLSを自動適用

7. 修正ロードマップ

Phase 1: デプロイ前必須（想定工数: 1-2日）

ID	概要	工数
C-1	<code>gsread.service_account()</code> に切替え（認証トークン自動更新）	30分
C-2	StateDB接続管理を単一コネクション+ <code>close()</code> に変更	1時間
F-01	サービスアカウントキーのパーミッション設定	2時間
F-04	SQLiteパスの絶対パス化（ <code>config</code> 設定追加）	2時間
F-03	シンボリックリンク追跡の制限	1時間
OH-5	Windowsサービス停止シグナル対応	1時間

Phase 2: 初回リリース後30日以内（想定工数: 3-5日）

ID	概要	工数
OH-1/6	Google API呼出しに指数バックオフリトライ追加	3時間
T-01	<code>sheets_sync.py</code> のモックテスト作成（8シナリオ以上）	4時間
T-02	<code>main.py</code> のモックテスト作成	4時間
F-05	ログの <code>spreadsheet_id</code> マスキング	1時間
OH-2	巨大Excelファイルサイズ上限チェック	30分
OH-7	壊れたExcelの連続失敗スキップ（サーキットブレーカー）	1時間
OH-8	シート全クリア後書き込み（余剰列対策）	30分
H-1/2	dataclass frozen=True化・イミュータビリティ改善	2時間

ID	概要	工数
H-4	バージョン文字列のsingle source of truth化	1時間

Phase 3: 継続的改善（90日以内）

ID	概要	工数
F-06	YAML型チェック・値域バリデーション強化	2時間
F-08	MD5 → SHA-256移行（+DBマイグレーション）	2時間
M-1/2	sync_workbook/sync_cycleの関数分割（50行以下に）	2時間
OM-7	ヘルスチェック・死活監視機構の実装	2時間
T-03/ 4/5	watcher/excel_readerのエッジケーステスト追加	3時間
その他	MEDIUM/LOW項目の順次対応	5時間

本レポートは 2026年03月26日 に静的コード分析により作成されました。
レビュー対象: tamatex v0.1.0 全ソースコード（src/tamatex/, scripts/, tests/）